```
require 'active_support'
require 'active_support/core_ext'
```

1.1.4 Loading All Active Support

And finally, if you want to have all Active Support available just issue:

```
require 'active_support/all'
```

That does not even put the entire Active Support in memory upfront indeed, some stuff is configured via `autoload`, so it is only loaded if used.

1.2 Active Support Within a Ruby on Rails Application

A Ruby on Rails application loads all Active Support unless `config.active_support.bare` is true. In that case, the application will only load what the framework itself cherry-picks for its own needs, and can still cherry-pick itself at any granularity level, as explained in the previous section.

2 Extensions to All Objects

2.1 `blank?` and `present?`

The following values are considered to be blank in a Rails application:

- `nil` and `false`,
- strings composed only of whitespace (see note below),
- empty arrays and hashes, and
- any other object that responds to `empty?` and is empty.

The predicate for strings uses the Unicode-aware character class `[:space:]`, so for example U+2029 (paragraph separator) is considered to be whitespace.

Note that numbers are not mentioned. In particular, 0 and 0.0 are **not** blank.

For example, this method from `ActionController::HttpAuthentication::Token::ControllerMethods` uses `blank?` for checking whether a token is present:

```
def authenticate(controller, &login_procedure)
  token, options = token_and_options(controller.request)
  unless token.blank?
    login_procedure.call(token, options)
  end
end
```

The method `present?` is equivalent to `!blank?`. This example is taken from `ActionDispatch::Http::Cache::Response`:

```
def set_conditional_cache_control!
  return if self["Cache-Control"].present?
  ...
end
```

Defined in `active_support/core_ext/object/blank.rb`.

2.2 `presence`

The `presence` method returns its receiver if `present?`, and `nil` otherwise. It is useful for idioms like this: